COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/DATA STRUCTURE AND ALGORITHMS	**SECTION:**	CPE21S2
DATE PERFORMED:	09/02/26	**DATE SUBMITTED:**	09/02/26
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA 6.1 Searching Algorithm

1. PROCEDURE OUTPUT

What procedures did you perform?

TABLE 6.1

SCREENSHOT	`g++ main.cpp -o main } ; if ($?) { .\main }`
	`12606 6381 22121 2577 3509 15908 30002 18341 14188 2278 28533 12926 11827 12758 13080 31103 15285 6400 10438 20`
	`98 27809 27244 5687 28674 9480 28914 30130 29099 21511 2895 26320 15086 2725 30585 7298 11401 31230 26672 27733`
	`15462 7330 6358 25333 18451 3896 21376 496 27506 23655 7501`

OBSERVATION	The observation is that the program successfully generated and displayed a large set of numbers in the console. The numbers appear in random order, showing that the program is working with multiple data values that can be used for searching or sorting operations.

TABLE 6.2A

CODE

```
6   template <typename T>
7   void linearSearchAR (int size, T dataFind, int array[]) {
8       // while loop
9       int i = 0;
10      // while loop implementation
11      while (i < size) {
12          if (dataFind = array[i]) {
13              std::cout << "Searching is successful!\n";
14              return;
15          }
16          i++;
17      }
18      std::cout << "Searching is unsuccessful!\n";
```

```cpp
#include <iostream>
#include <cstdlib> // for generating random integers
#include <time.h> // will be used for our seeding function
#include "searchingAlgo.h"
#include "node.h"

const int max_size = 50;

int main() {

    int dataset[max_size];
    srand (time(0));
    for (int i = 0; i <max_size; i++) {
        dataset[i] = rand();
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    std::cout << "What number to find: ";
    int choice;
    std::cin >> choice;

    linearSearchAR(max_size, choice, dataset);
```

OUTPUT

```
PS C:\Users\pagan\OneDrive\Documents\Bungag 6p1> cd "c:\Users\pagan\OneDrive\Documents\Bungag 6p1\" ; if
($?) { g++ main.cpp -o main } ; if ($?) { .\main }
24924 15354 1999 2036 14467 16398 6455 19476 3939 11511 27687 15230 26802 2460 32445 25454 17940 5416 186
3 1410 29942 28765 18511 11191 8890 28972 26069 14484 31765 21634 8155 18341 23478 16996 5671 29227 19497
 22590 31044 19474 24826 11640 19658 11555 11037 1611 7903 6779 19597 10451
What number to find: 18341
Searching is successful!
PS C:\Users\pagan\OneDrive\Documents\Bungag 6p1>
```

OBSERVATION

The program successfully generated 50 random numbers and displayed them in the console. It then allowed the user to enter a number to search for using the sequential search algorithm. In the given output, the program searched for 18341 and displayed that the search was successful. However, the code uses = instead of == in the if condition, so the program is assigning a value rather than properly comparing the numbers. This should be corrected to make sure the search result is accurate.

TABLE 6.2B

CODE

```cpp
template <typename T>
void linearSearchLL (Node<T> *nodes, T dataFind) {
    // repeat the loop until the correct node is not NULL
    while (nodes != nullptr) {
        // check if the item = current node data
        if (nodes->data == dataFind) {
            std::cout << "The data: " << nodes->data << " was found successfully!\n";
            return;
        }
        // move on to the next node
        nodes = nodes->next;
    }
    // display unsuccessful
    std::cout << "Searching is unsuccessful!\n";
}
```

```cpp
    // CREATE A NODE FOR YOUR NAME

    // CREATE A NODE
    Node<char> *char1 = new_node('M');
    Node<char> *char2 = new_node('A');
    Node<char> *char3 = new_node('R');
    Node<char> *char4 = new_node('C');

    // CONNECT THE NODES
    char1->next = char2;
    char2->next = char3;
    char3->next = char4;
    char4->next = nullptr;

    linearSearchLL(char1, 'R');
```

OUTPUT

```
PS C:\Users\pagan\OneDrive\Documents\Bungag 6p1> cd "c:\Users\pagan\OneDrive\Documents\Bungag 6p1\" ; if ($?) { g++ main.cpp -o ma
in } ; if ($?) { .\main }
The data: R was found successfully!
```

OBSERVATION

The program successfully created a linked list containing the characters M → A → R → C and performed a sequential search for the character R. The search checked each node one by one until it reached R, then displayed that the data was found successfully. This shows that the linked list sequential search works by starting from the first node and moving to the next node until the target value is found

TABLE 6.3A

CODE

```
// generate a sorted array: (use for loop)
int newArr[max_size];
for (int i = 0; i < max_size; i++) {
    newArr[i] = i;
}

binarySearchArr (newArr, max_size, 11);
```

```
template <typename T>
void binarySearchArr (T data[], int arrSize, T dataFind) {
    int low = 0, high = arrSize - 1, middle;
    while (low <= high) {
        middle = (int)((low + high / 2));
        if (dataFind = data[middle]) {
            std::cout << "The data was found!\n";
            return;
        }
        else if (dataFind < data[middle]) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }
    std::cout << "The data was not found!\n";
}
```

OUTPUT

```
PS C:\Users\pagan\OneDrive\Documents\Bungag 6p1> cd "c:\Users\pagan\OneDrive\Documents\Bungag 6p1\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
The data was found!
PS C:\Users\pagan\OneDrive\Documents\Bungag 6p1>
```

OBSERVATION

The program successfully created a sorted array and applied the binary search algorithm to look for the value 11. The algorithm repeatedly divides the search range in half and compares the middle value with the target. The output shows that the data was found successfully, demonstrating that binary search can efficiently locate an element in a sorted array

TABLE 6.3B

CODE

```cpp
// middle function getter:
template <typename T>
Node<T>* getMiddle (Node<T>* start, Node<T>* last) {

    Node<T>* slow = start;
    Node<T>* fast = start;

    // continue until fast reaches the end or the last node of the current search range
    while (fast != last && fast->next != last) {
        // move slow one step
        slow = slow->next;
        // move fast two steps
        fast = fast->next->next;
    }
    return slow; // return middle
}

template <typename T>
void binarySearchLL (Node<T>* start, T dataFind) {
    // step 1: Set start to the head of the list and set last to null
    Node<T>* last = nullptr;

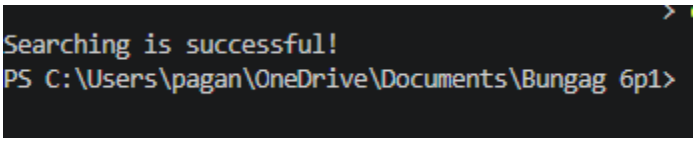
    while (start != last) {
        // middle element is calculated using the two pointers approach discussed above
        Node<T>* middle = getMiddle (start, last);
        // if the middle element is same as the key to be sorted, we return it
        if (middle->data == dataFind) {
            std::cout << "Searching is successful!\n";
            return;
        }
        else if (middle->data > dataFind) {
            last = middle;
        }
        else {
            start = middle->next;
        }
    }
    std::cout << "Searching is unsuccessful!\n";
}

#endif
```

```cpp
// CREATE A NODE*/
Node<char> *char1 = new_node('M');
Node<char> *char2 = new_node('A');
Node<char> *char3 = new_node('R');
Node<char> *char4 = new_node('C');

// CONNECT THE NODES
char1->next = char2;
char2->next = char3;
char3->next = char4;
char4->next = nullptr;
```

```cpp
binarySearchLL (char1, 'A');
```

OUTPUT	
OBSERVATION	The program successfully created a linked list containing the characters M → A → R → C and used the binary search algorithm to search for the character A. The algorithm uses two pointers, slow and fast, to find the middle node of the current search range and then narrows the search based on the target value. This demonstrates how binary search can also be implemented using a linked list, although it is generally more complex than using an array because the middle element must be located by traversing the nodes

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM 1

CODE

supplementary.h > linearSearchAR<T>(int, T, T [])

```
#ifndef SUPPLEMENTARY_H
#define SUPPLEMENTARY_H

#include <iostream>
#include "node.h"

// Linear Search - Array
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]) {
    int comparisons = 0;

    for (int i = 0; i < size; i++) {
        comparisons++;

        if (dataFind == array[i]) {
            std::cout << "Searching is successful!\n";
            std::cout << "Data found: " << array[i] << "\n";
            std::cout << "Comparisons: " << comparisons << "\n";
            return;
        }
    }

    std::cout << "Searching is unsuccessful!\n";
    std::cout << "Comparisons: " << comparisons << "\n";
}

```

```cpp
// Linear Search - Linked List
template <typename T>
void linearSearchLL(Node<T>* nodes, T dataFind) {
    int comparisons = 0;

    while (nodes != nullptr) {
        comparisons++;

        if (nodes->data == dataFind) {
            std::cout << "Searching is successful!\n";
            std::cout << "Data found: " << nodes->data << "\n";
            std::cout << "Comparisons: " << comparisons << "\n";
            return;
        }

        nodes = nodes->next;
    }

    std::cout << "Searching is unsuccessful!\n";
    std::cout << "Comparisons: " << comparisons << "\n";
}

#endif
```

OUTPUT

```
===== ARRAY LINEAR SEARCH =====
Key: 18
Searching is successful!
Data found: 18
Comparisons: 2

===== LINKED LIST LINEAR SEARCH =====
Key: 18
Searching is successful!
Data found: 18
Comparisons: 2
```

ANALYSIS

The program successfully implements sequential search for both an array and a linked list. When searching for the key 18, both approaches found the value successfully after 2 comparisons, since 18 is the second element in the given list. The array search checks each element using an index, while the linked list search moves from one node to the next until the key is found. The output confirms that both implementations produced the same result, showing that sequential search works correctly for both data structures.

PROBLEM 2

CODE

```cpp
//============================PROBLEM 2=================================
// Sequential Search - Array
template <typename T>
int linearSearchAR(int size, T dataFind, T array[]) {
    int count = 0;

    for (int i = 0; i < size; i++) {
        if (array[i] == dataFind) {
            count++;
        }
    }

    return count;
}

// Sequential Search - Linked List
template <typename T>
int linearSearchLL(Node<T>* nodes, T dataFind) {
    int count = 0;

    while (nodes != nullptr) {
        if (nodes->data == dataFind) {
            count++;
        }

        nodes = nodes->next;
    }

    return count;
}
#endif
```

```cpp
//============================PROBLEM 2=================================
    int array[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;

    // Array
    int arrayCount = linearSearchAR(size, key, array);

    std::cout << "===== ARRAY =====\n";
    std::cout << "Search key: " << key << "\n";
    std::cout << "Number of repeating instances: " << arrayCount << "\n";


    // Linked List
    Node<int>* head = nullptr;
    Node<int>* tail = nullptr;

    for (int i = 0; i < size; i++) {
        Node<int>* newNode = new_node(array[i]);

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        }
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }

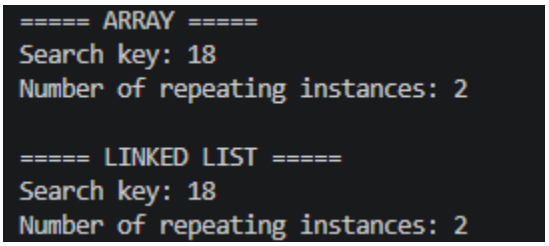
    int listCount = linearSearchLL(head, key);

    std::cout << "\n===== LINKED LIST =====\n";
    std::cout << "Search key: " << key << "\n";
    std::cout << "Number of repeating instances: " << listCount << "\n";


    // Delete linked list
    Node<int>* current = head;

    while (current != nullptr) {
        Node<int>* temp = current;
        current = current->next;
        delete temp;
    }

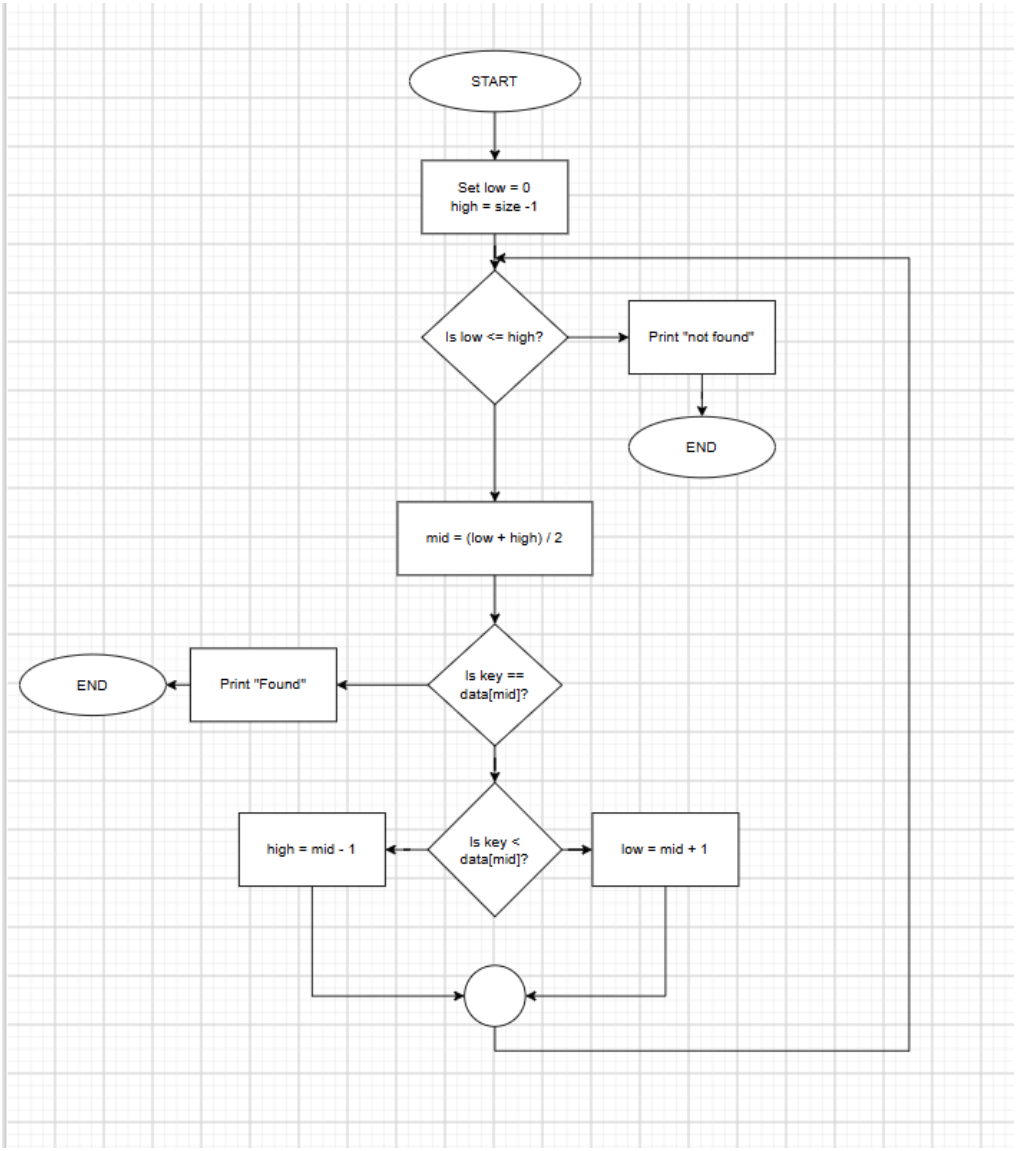
    return 0;
}
```

OUTPUT	
ANALYSIS	The program successfully counts the repeating instances of the search key 18 in both the array and linked list. Since 18 appears twice in the given list, both search methods correctly return a count of 2. The array searches each element using its index, while the linked list checks each node and moves to the next node until it reaches the end. The matching output from both methods shows that the modified sequential search works correctly for counting repeated values.

PROBLEM 3

ALGORITHM	1. Start. 2. Set low = 0. 3. Set high = size - 1. 4. Check if low <= high. 5. If No, display "Not Found" and end. 6. If Yes, calculate mid = (low + high) / 2. 7. Check if key == data[mid]. 8. If Yes, display "Found" and end. 9. If No, check if key < data[mid]. 10. If Yes, set high = mid - 1. 11. If No, set low = mid + 1. 12. Repeat steps 4–11 until the key is found or low > high. 13. End.

DIAGRAM


START
Set low = 0
high = size -1
Is low <= high?
Print "not found"
END
mid = (low + high) / 2
Is key == data[mid]?
Print "Found"
END
Is key < data[mid]?
high = mid - 1
low = mid + 1

CODE

```cpp
// ==================== PROBLEM 3 ====================

// Binary Search - Array
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind) {
    int low = 0;
    int high = arrSize - 1;
    int middle;
    int comparisons = 0;

    while (low <= high) {
        middle = (low + high) / 2;
        comparisons++;

        if (dataFind == data[middle]) {
            std::cout << "The data was found!\n";
            std::cout << "Data found: " << data[middle] << "\n";
            std::cout << "Comparisons: " << comparisons << "\n";
            return;
        }
        else if (dataFind < data[middle]) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }

    std::cout << "The data was not found!\n";
    std::cout << "Comparisons: " << comparisons << "\n";
}
#endif
```

```
98      // ==================== PROBLEM 3 ====================
99
100     int array[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
101     int size = 10;
102     int key = 8;
103
104     std::cout << "===== BINARY SEARCH - ARRAY =====\n";
105     std::cout << "Search key: " << key << "\n";
106
107     binarySearchArr(array, size, key);
108
109     return 0;
110 }
```

OUTPUT

```
===== BINARY SEARCH - ARRAY =====
Search key: 8
The data was found!
Data found: 8
Comparisons: 4
```

ANALYSIS

The program successfully applies the binary search algorithm to the sorted array [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] to find the key 8. Instead of checking every element one by one, the algorithm repeatedly checks the middle value and reduces the search area based on whether the key is smaller or larger. The search checks 11, then 5, then 6, and finally finds 8 at index 3 after 4 comparisons. This shows that binary search can efficiently find a value in a sorted array by continuously dividing the search range.

PROBLEM 4

CODE

```
// Recursive Binary Search
template <typename T>
int binarySearchRecursive(T data[], int low, int high, T dataFind, int& comparisons) {

    // Check if the search range is still valid
    if (low > high) {
        return -1;
    }

    // Find the middle element
    int middle = (low + high) / 2;
    comparisons++;

    // Check if the key is found
    if (data[middle] == dataFind) {
        return middle;
    }

    // Search the left half
    if (dataFind < data[middle]) {
        return binarySearchRecursive(data, low, middle - 1, dataFind, comparisons);
    }

    // Search the right half
    return binarySearchRecursive(data, middle + 1, high, dataFind, comparisons);
}


#endif
```

```cpp
    // Problem 3 list
    int array[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int size = 10;
    int key = 8;
    int comparisons = 0;

    std::cout << "===== RECURSIVE BINARY SEARCH =====\n";
    std::cout << "Search key: " << key << "\n";

    int result = binarySearchRecursive(
        array,
        0,
        size - 1,
        key,
        comparisons
    );

    if (result != -1) {
        std::cout << "The data was found!\n";
        std::cout << "Data found: " << array[result] << "\n";
        std::cout << "Index: " << result << "\n";
        std::cout << "Comparisons: " << comparisons << "\n";
    }
    else {
        std::cout << "The data was not found!\n";
        std::cout << "Comparisons: " << comparisons << "\n";
    }

    return 0;
}
```

OUTPUT

```
===== RECURSIVE BINARY SEARCH =====
Search key: 8
The data was found!
Data found: 8
Index: 3
Comparisons: 4
```

ANALYSIS

The program successfully modifies the binary search algorithm into a recursive function and uses it to search for the key 8 in the sorted array. Instead of using a loop, the function repeatedly calls itself while reducing the search range based on the middle value. The search checks 11, then 5, then 6, and finally finds 8 at index 3. The result shows 4 comparisons, proving that the recursive version produces the same correct result as the original binary search while using recursion to perform the search.

CONCLUSION:
In this activity, I learned how different searching algorithms work using both arrays and linked lists. I learned that sequential search checks each element one by one, while binary search is faster because it repeatedly divides a sorted list into smaller parts. I also learned how to modify sequential search to count repeating values and how to implement binary search using recursion. Overall, these activities helped me understand how choosing the right searching method can make finding data easier and more efficient depending on the type and arrangement of the data

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.